

Documentación técnica

Integrantes

1. Manuel Politis Kufa
2. Gonzalo Alejo Dominguez
3. Ignacio Zanon Mutti

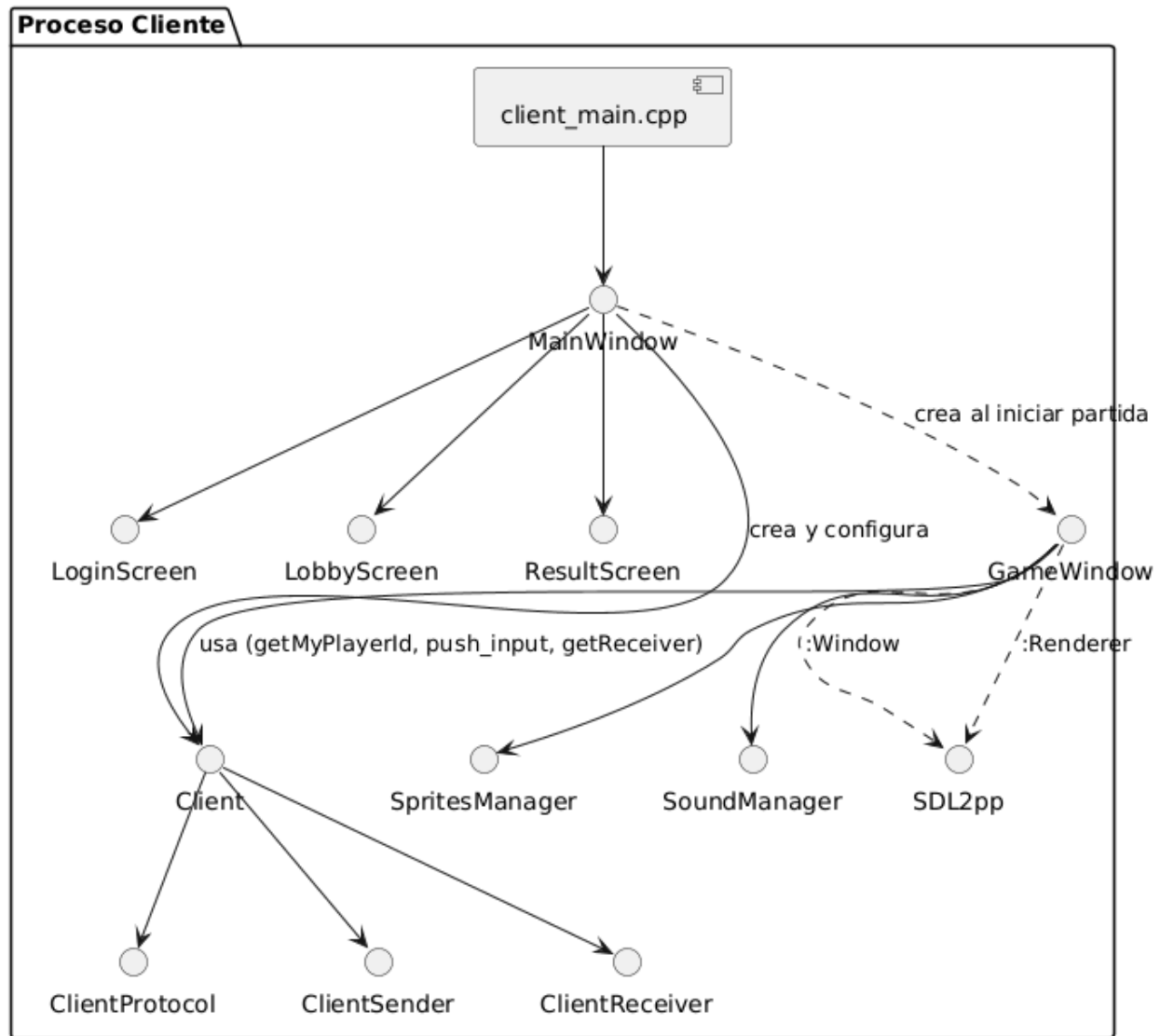
Cliente

1. Arquitectura general del cliente

El cliente se divide en tres grandes bloques:

- **Qt:** pantallas de login, lobby y resultados.
- **Conexión:** conexión TCP, protocolo binario, hilos de envío/recepción y colas.
- **Juego SDL:** ventana de juego, renderizado de mapa, autos, HUD y sonidos.

1.1 Diagrama de arquitectura

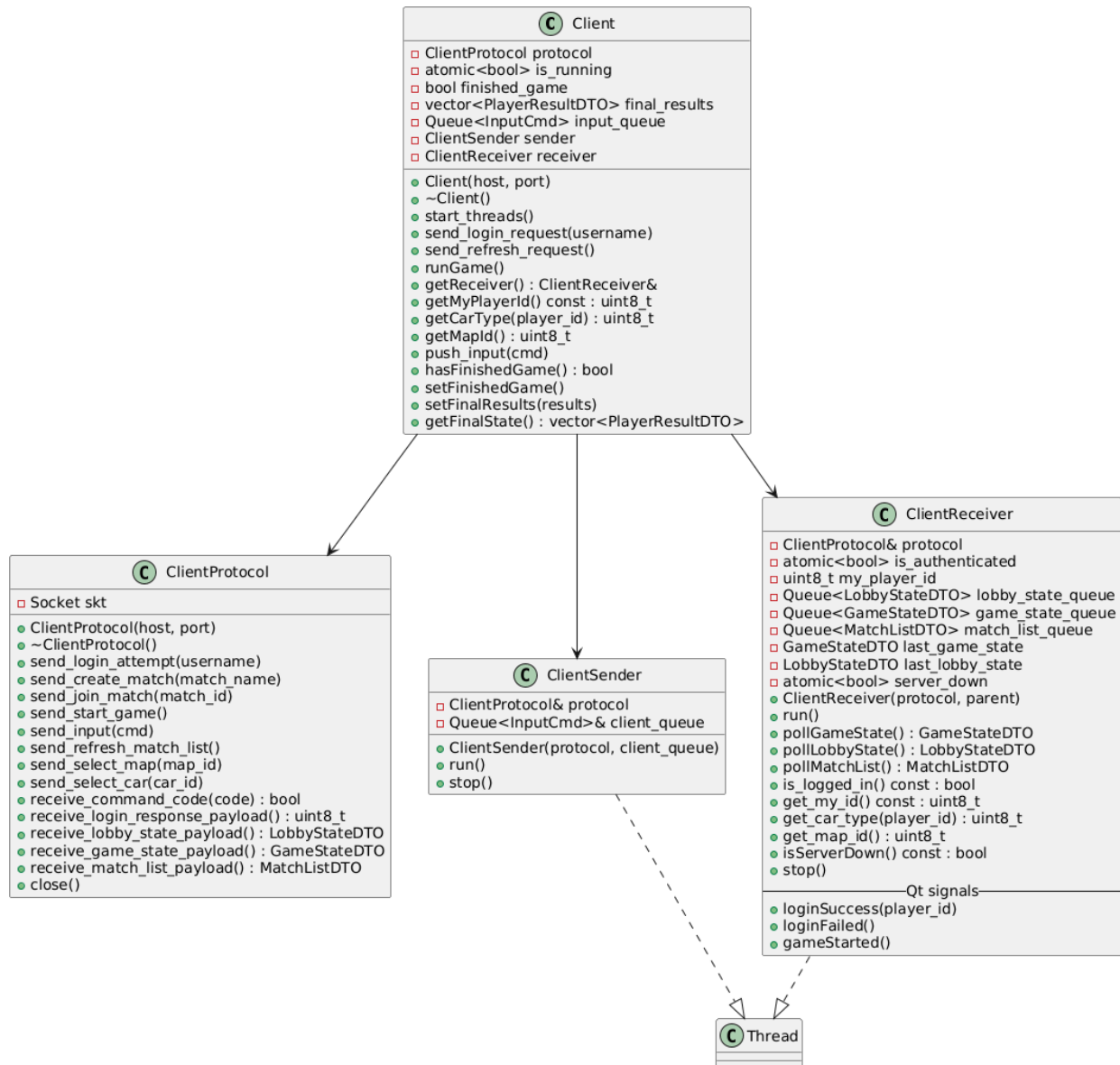


- `client_main.cpp` levanta Qt, crea `MainWindow` y se queda en el loop de eventos.
- `MainWindow` se encarga de:
 - manejar las pantallas (`LoginScreen`, `LobbyScreen`, `ResultScreen`)
 - crear el `Client` con el host/puerto
 - lanzar la `GameWindow` cuando el servidor avisa que empieza la partida.

- 
- `Client` encapsula la lógica de red:
 - `ClientSender` (hilo que saca `InputCmd` de la cola y los manda por socket).
 - `ClientReceiver` (hilo que escucha el socket y mete estados en colas).
 - `GameWindow` usa `SDL2pp` para dibujar la carrera y `SoundManager` para el audio.
-

2. Socket y Threads

2.1 Diagrama de clases (Client + protocolo + threads)

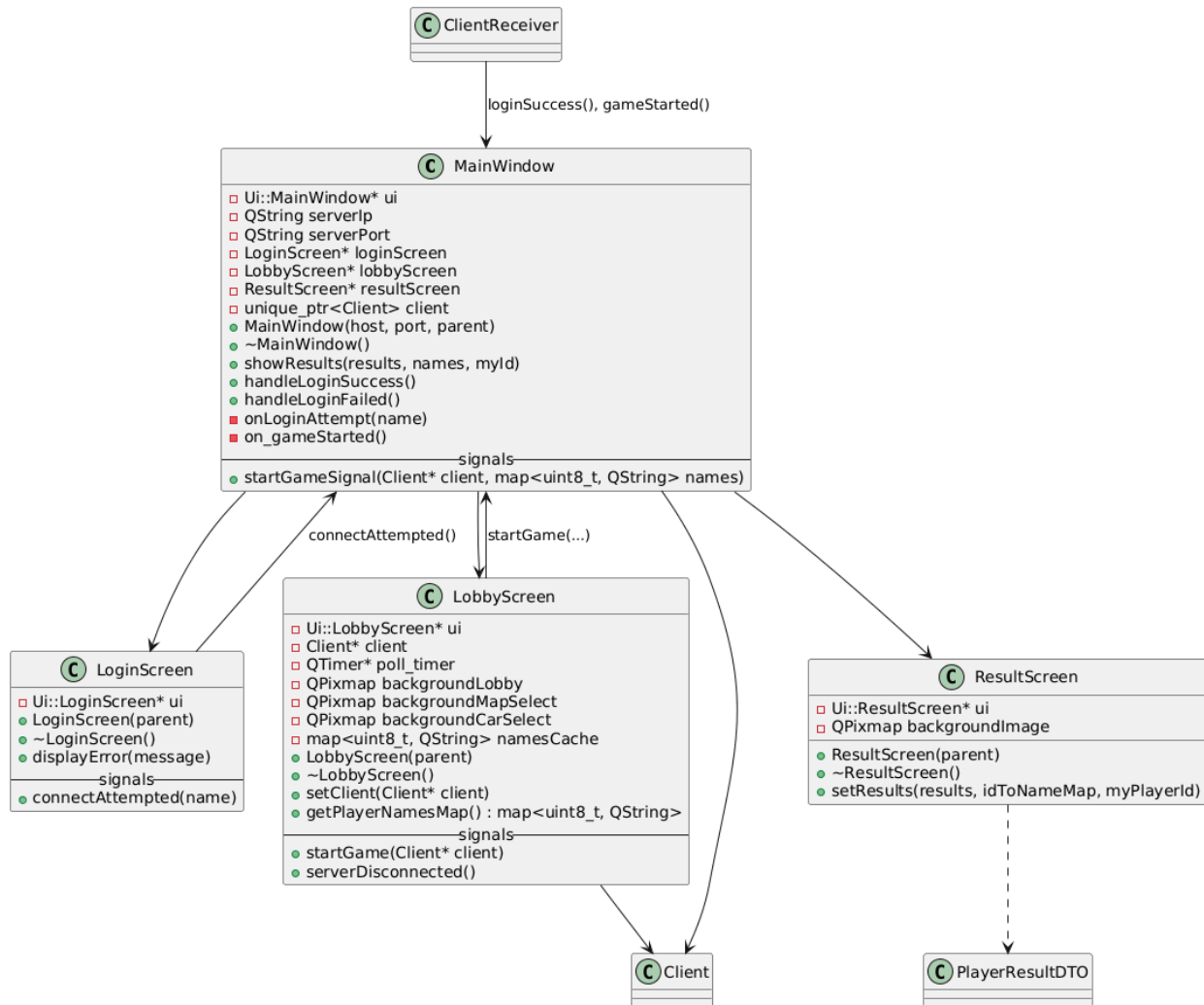


- **ClientProtocol** es la única clase que toca el **Socket**:
 - serializa comandos (login, crear/unirse partida, inputs, selección de mapa/auto, start game).

- deserializa respuestas (estado de lobby, lista de partidas, estado de juego, resultados).
 - **ClientSender** corre en un hilo:
 - hace `pop()` sobre `Queue<InputCmd>` y llama a los métodos de `ClientProtocol`.
 - reutiliza la misma cola para comandos de lobby y comandos de juego.
 - **ClientReceiver** corre en otro hilo:
 - lee un byte de **código de comando** y decide qué payload leer.
 - actualiza las colas de `LobbyStateDTO`, `GameStateDTO`, `MatchListDTO`.
 - expone métodos `poll*` que devuelven el último estado conocido (usando `last_*`).
 - **Client** junta todo:
 - arranca/detiene los threads,
 - ofrece métodos de conveniencia a las capas superiores (Qt/SDL).
-

3. Interfaz gráfica Qt

3.1 Diagrama de clases Qt



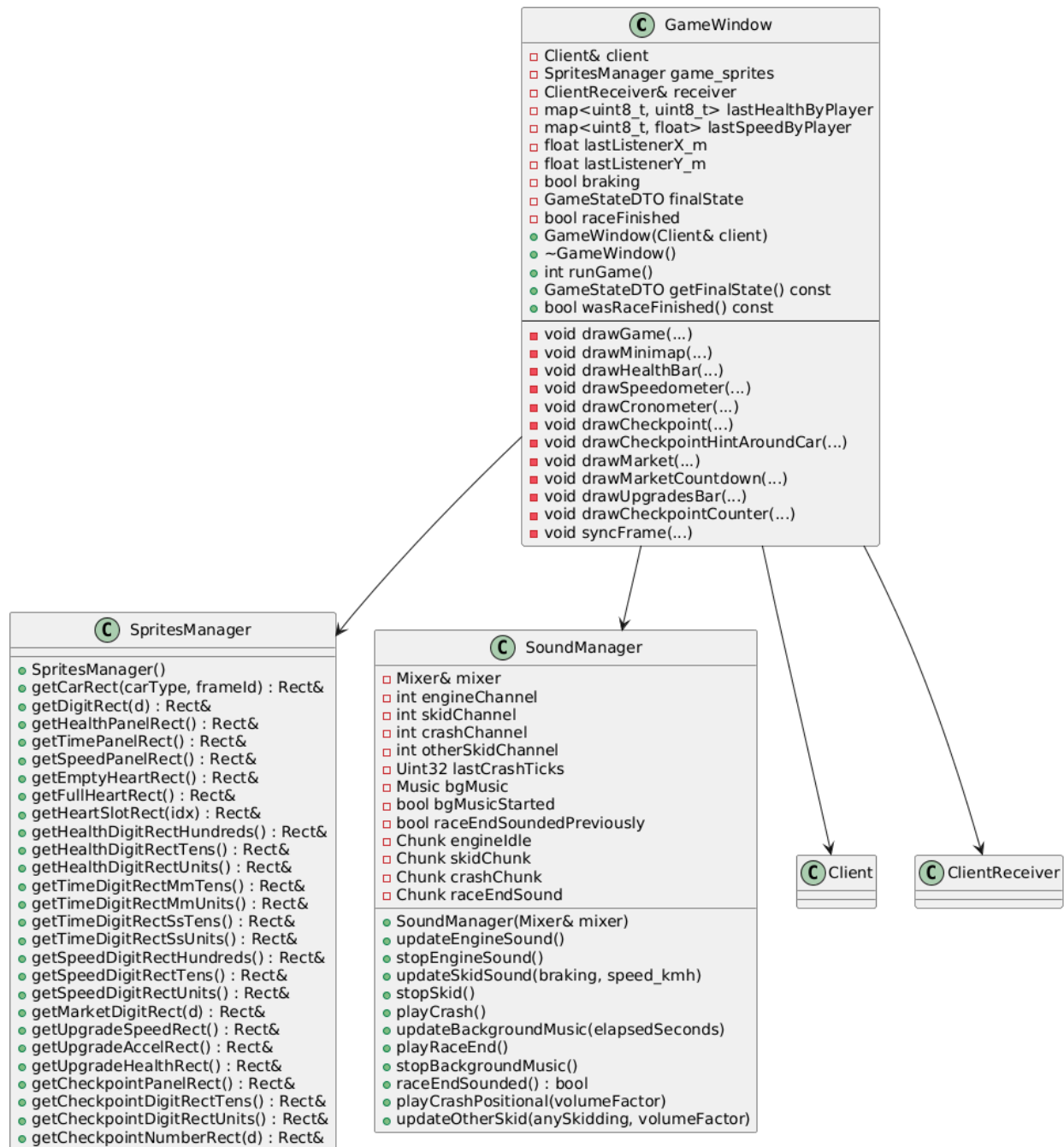
- **MainWindow:**

- crea las tres pantallas y las maneja con un `QStackedWidget`.
- gestiona el ciclo: Login → Lobby → Juego SDL → Resultados.
- maneja las señales de `ClientReceiver`: en `loginSuccess` cambia al lobby, en `gameStarted` emite `startGameSignal`.

- 
- **LoginScreen:**
 - muestra el fondo de login y un campo de nombre de usuario.
 - emite `connectAttempted(name)` cuando el usuario pulsa “Conectar”.
 - **LobbyScreen:**
 - usa un `QTimer` para llamar periódicamente a `client->getReceiver().pollLobbyState()` y `pollMatchList()`.
 - permite crear/join partidas, seleccionar mapa, seleccionar auto y ver a los jugadores conectados.
 - convierte acciones de UI en `InputCmd` y los envía con `client->push_input`.
 - **ResultScreen:**
 - muestra una tabla de hasta 8 jugadores con posición y tiempo.
 - resalta al jugador local y cambia el label a “¡VICTORIA!” o “HAS PERDIDO :(”.
-

4. Juego y renderizado SDL

4.1 Diagrama de clases SDL (GameWindow + sprites + audio)



- `runGame()`:
 - inicializa SDL (video + audio) y crea `Window`, `Renderer` y `Mixer`.
 - carga todas las texturas (autos, HUD, mapa, market, etc.) y sonidos.
 - llama a `drawGame(...)`, que contiene el loop principal.
- En cada frame, `drawGame`:
 - llama a `receiver.pollGameState()`.
 - Si `race_finished == 1`, detiene sonidos, guarda resultados en `Client` y sale.
 - procesa eventos SDL (teclado y cierre de ventana):
 - genera `InputCmd` y los manda al servidor a través de `client.push_input`.
 - controla cuándo se permite comprar mejoras (solo durante tiempo de compra).
 - actualiza la cámara para seguir al auto del jugador local.
 - dibuja el fondo, autos, mini-mapa, HUD (vida, tiempo, velocidad, checkpoints, upgrades).
 - si el jugador local está muerto, dibuja overlay “moriste”.
 - actualiza audio con `SoundManager`:
 - motor en loop,
 - skid propio y de otros con atenuación por distancia,
 - golpes (con cooldown), música de fondo y sonido de final de carrera.
 - llama a `syncFrame` para mantener 60 FPS estables.
- `SpritesManager`:
 - centraliza todos los `Rect` de los sprites: autos, dígitos, paneles, corazones, etc.
- `SoundManager`:
 - encapsula el uso de `SDL2pp::Mixer`:
 - reproducir loops (motor, derrape),
 - reproducir efectos (choques, final),
 - ajustar volumen según la distancia y estado del juego.

Protocolo

Cada mensaje consiste en:

- Comando (uint8_t)
- Payload (longitud variable según el comando)

Todos los datos se envían en formato binario utilizando:

- uint8_t
- uint16_t en big endian (htons, ntohs)
- float pasado a uint32_t y enviado también en big endian

Comandos enviados por el Cliente:

Comando	Código	Payload	Descripción
LOGIN	0x01	[uint16_len][username]	Solicita iniciar sesión
CREATE_MATCH	0x05	[uint16_len][name]	Crea una partida
JOIN_MATCH	0x06	[match_id]	Se une a una partida existente
START_GAME	0x07	-	Indica al host iniciar la partida
REFRESH_MATCH_LIST	0x08	-	Solicita lista de partidas
SELECT_MAP	0x09	[map_id]	Selección de mapa

SELECT_CAR	0x0A	[car_id]	Selección de auto
SEND_INPUT	0x04	[action][key][player_id]	Inputs de de teclado del jugador

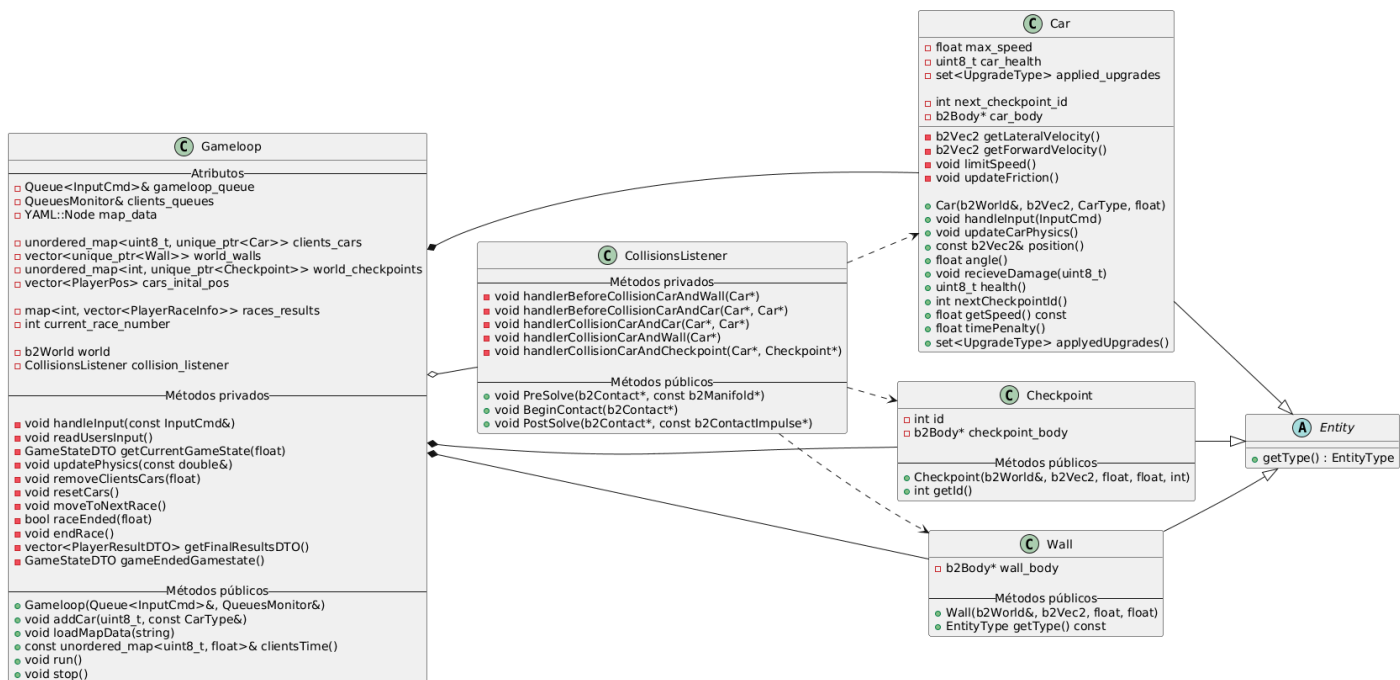
Respuestas / Eventos enviados por el Servidor

Evento	Código	Payload	Descripción
LOGIN_SUCCESS	0x02	[player_id]	Login exitoso
LOGIN_FAILED	0x03	-	Login rechazado
RSP_MATCH_LIST	0x92	Lista de partidas	Retorna partidas disponibles
RSP_LOBBY_STATE	0x91	Estado del lobby	Jugadores, host, mapa, autos
EVT_GAME_STARTED	0x70	-	Señal de inicio
EVT_GAME_STATE	0x80	Estado completo del juego	Posiciones, velocidad, upgrades, checkpoints

Lógica del Servidor

A nivel lógica de juego la clase más importante de todas es la clase Gameloop. Por ella pasan todos los comandos recibidos de parte de los clientes y es la que maneja el flujo de la partida y que es lo que ocurre en general dentro del juego.

Servidor - Lógica de Juego



Clase Gameloop

Esta clase es la representación de toda una partida y sus carreras por lo que dentro del servidor va a haber una instancia de esta clase corriendo en un hilo por cada partida que se haya creado.

Es responsable de:

- 
- procesar los inputs de los jugadores
 - Cargar la información de los archivos de los mapas
 - actualizar la simulación física mediante el motor Box2D
 - administrar el estado de los autos (vida, velocidad, checkpoints, destrucción)
 - detectar colisiones y aplicar sus efectos
 - generar la actualización del estado del mundo
 - Seguir el progreso de los jugadores a lo largo de las carreras
 - Determinar el ranking final
 - enviarlo a cada cliente a través de colas thread-safe

La clase del servidor *Match* que es la representación de una partida activa va a crear una instancia del Gameloop. Lo que permite soportar múltiples partidas simultáneas sin problemas.

Componentes

La clase Gameloop mantiene las siguientes estructuras principales:

- `clients_cars: Map<uint8_t, unique_ptr<Car>>`
Mapa que asocia cada ID de jugador con su auto correspondiente.
- `world_walls: Vector<unique_ptr<Wall>>`
Muros del mapa cargados desde el archivo correspondiente a el mapa en particular
- `world_checkpoints: Map<int, unique_ptr<Checkpoint>>`
Mapa que asocia un checkpoint a su ID correspondiente y que marca la progresión de la carrera.
- `collision_listener: CollisionsListener`
Parte de Box2d que capta colisiones físicas y aplica lógica de daño y detección de checkpoints.
- colas de comunicación:
Se reciben `InputCmd` desde los jugadores desde la cola `gameloop_queue` y se envían `GameStateDTO` con un método `broadcast GameStateDTO` a cada uno de los jugadores.

Ciclo Principal en Run()

El gameloop corre un loop principal dentro de este metodo a un ritmo constante de 60 cuadros por segundo usando un ciclo del tipo drop&rest por lo que se garantiza que siempre dure lo mismo.

En el mismo se hace lo siguiente:

- Se leen y procesan todos los inputs pendientes
- Se actualizan las fuerzas que se aplican a los autos y se hace avanzar un ciclo a la simulacion del mundo en Box2d
- Se eliminan los clientes que hayan terminado la carrera actual o hayan muerto en la misma
- Se chequea si termino la carrera
- Si no termina se envía a todos los clientes el estadio actual de la partida (Posiciones de los jugadores, vida, mejoras compradas, sig checkpoint, etc.)
- Se mantiene el tiempo del ciclo

Cuando la carrera termina, se avanza a la siguiente con ayuda de métodos como resetCars, loadMapData, etc. que dejan todo listo para la sig carrera.

Entidades del Mapa

Está la clase abstracta *Entity* que representa una entidad del mundo de Box2d, de la misma heredan car, wall y checkpoint.

El método que todos comparten en común es EntityType que cada clase sabe responder de forma diferente. Esto es utilizado en el componente de colisiones CollisionHandler, el que Box2d envía los elementos del mapa a priori sin decir de qué tipo son por lo que este metodo es util para esa tarea y saber de qué tipo de colisión se trata y cuales son sus consecuencias

Car

Entidad dinámica controlada por el jugador:

- Posee un b2Body dinámico.
- Contiene estado de control (acelerar, frenar, girar).
- Aplica fricción lateral, límites de velocidad y lógica de manejo arcade.

- Mantiene información de la carrera (vida, checkpoint actual, upgrades aplicadas).
- Tiene la responsabilidad de saber cómo moverse a sí mismo dentro del mapa.

Wall

Entidad estática:

- Modelada como un b2Body estático rectangular.
- Define colisiones sólidas del mapa (paredes y límites).

Checkpoint

Entidad sensor:

- No produce colisiones físicas.
- Sirve para detectar paso de los autos.
- Posee un ID que define el orden de progresión de la carrera.

Sistema de Colisiones (CollisionListener)

El servidor utiliza una clase que hereda de b2ContactListener provisto por Box2D para la detección de colisiones.

En base a lo que responden los objetos recibidos al metodo entityType, determina si la colision es auto con auto, auto con checkpoint o auto con pared

En *Presolve*, que es la etapa previa a la resolución de la colisión, se guardan las velocidades de los autos involucrados.

En *beginContact* se le presta atención a las colisiones con los checkpoints

- Se verifica si coincide con su next_checkpoint_id.
- Si es correcto, avanza al siguiente.



En *PostSolve* se compara la velocidad guardada en *presolve* con la actual y con ese delta determinar la fuerza del impacto.

Si el valor supera un umbral configurable:

- Se calcula daño proporcional.
- Se reduce la salud del auto.

Toda la lógica de colisiones se encapsula acá y es la que después utiliza el world dentro de la clase gameloop en la simulación física.